

CUSTOMER ENGINEERING · CONFIDENTIAL · JULY 2026

Real-Time Fraud Detection on Snowflake

Proof of Concept — Prepared for Zilch CTO

Online Feature Store

SPCS Scoring Service

Snowpark ML

Model Registry

Card testing attacks complete in seconds. A model on stale features approves every transaction.

Fraudsters fire **5–10 rapid transactions in under 30 seconds** to validate stolen card credentials before the card is blocked. If your model only sees yesterday's velocity data, every transaction in the burst looks clean — until the attack is over.

Txn 1	Txn 2	Txn 3	Txn 4	Txn 5	Chargeback
APPROVED	APPROVED	APPROVED	APPROVED	APPROVED	TOO LATE

All 5 transactions approved in <30s. Fraud confirmed after the fact.

This is not a model accuracy problem.

The same model, same weights, same threshold — only feature freshness changes. Stale features make the burst invisible.

THE LOSSES GO BEYOND THE FRAUD SPEND

- ▶ Chargebacks — reversed even after approval
- ▶ Issuer fees on every chargeback dispute
- ▶ Reputational risk with Thredd
- ▶ Engineering cost to build detection from scratch

ESTIMATED EXPOSURE AT CURRENT VOLUME

- 24M transactions/yr · 0.05% fraud rate = 12,000 incidents/yr
- Card-testing estimated at 20% of fraud volume
- **~£1.2M/yr** directly attributable to burst attacks on stale features

The foundation is solid. One missing connection is costing you detections.

WHAT'S WORKING TODAY

✓ XGBoost model in production

Kinesis → SageMaker → DynamoDB →
Triton — fraud scoring is live

✓ Transaction data already in Snowflake

The data foundation is in place —
Snowflake is not a migration

THE THREE GAPS

⚠ Feature freshness not yet built

Burst detection window undefined — card-
testing transactions look clean until the
pipeline catches up

⚠ Training and serving in separate stacks

SageMaker/RAPIDS training vs
Triton/DynamoDB serving — feature drift
risk on every retrain without careful
alignment

⚠ 10 services across 3 AWS families

Each scales, fails, and deploys
independently — incidents require multiple
teams to isolate root cause

This is a targeted gap, not a wholesale replacement. The XGBoost model works. The transaction data is already in Snowflake. What's missing is the real-time feature pipeline that connects them — and that is exactly what this proof of concept delivers.

End-to-end real-time fraud scoring — **deployed and benchmarked on live infrastructure.**

Payment Gateway (Thredd event) → Zilch EHI

AWS PRIVATELINK · ASYNC

→ OFS Ingest API · 7-field event · features in <2s

AWS PRIVATELINK · SYNC

→ SPCS Scoring Endpoint · returns approve / flag / block

THREAD A · ASYNC

Snowpipe Streaming

Full transaction record →

FRAUD_TRANSACTIONS

Exactly-once persistence. Zero latency on auth path. Audit trail and training data.

THREAD B · EHI ASYNC

OFS REST Ingest (direct)

EHI fires this over PrivateLink — not through SPCS.

Thin 7-field event → 4

CONTINUOUS velocity pipelines

Customer · Merchant · DPAN · IP velocity features live in <2s.

Does not touch the scoring path.

THREAD C · SYNC

Score & Decide

Feature Group query (1 call, all 5 FVs) → ~11ms

Derived features inline →

~0.1ms

XGBoost.predict(46 features) →

~1ms

Preliminary Zilch POC: ~20–40ms avg end-to-end · Returns: approve / flag / block

Snowpipe
Streaming

Online
Feature Store

Snowpark ML
Training

Model
Registry

SPCS
Scoring Service

Same outcome. Radically simpler architecture.

Layer	Zilch Today — AWS Native	Snowflake
Ingestion	Kinesis + Lambda + S3	Snowpipe Streaming (SPCS async) + OFS REST Ingest (EHI direct via PrivateLink)
Feature engineering	Glue ETL jobs + RAPIDS preprocessing	None — declare features once at registration. No ETL.
Online feature store	Batch snapshots — minutes stale	CONTINUOUS aggregation — <2s freshness out of the box
Model training	SageMaker + ECR + S3 data staging	Snowpark-Optimized warehouse — ~5 min/run, ~0.5 credits
Model serving	Triton + fraud-engine-service + EKS	SPCS ingress endpoint — accessed via PrivateLink from EHI. CREATE SERVICE (one command) + one VPC Interface Endpoint in Zilch's AWS account.
Training/serving consistency	Serialized encoders from S3 — version drift risk on every retrain	Arithmetic inline — identical logic, drift structurally impossible
Monitoring	CloudWatch + X-Ray + custom dashboards per service	Unified query history + Snowflake Model Monitor
Compliance boundary	Data crosses multiple AWS service boundaries	Data never leaves Snowflake's network
Network connectivity	VPC security groups + NAT + public API Gateway endpoints	AWS PrivateLink — one interface VPC Endpoint resolves *.snowflakecomputing.app to a private IP. Covers OFS Ingest + SPCS scoring in a single endpoint config. No public internet.
Services to operate	10+	1 platform
Time to production	Months (batch fallback planned for MVP due to real-time complexity)	Weeks — real-time path is the MVP path. Proven in this POC.

All success criteria met on live infrastructure. These are measured numbers, not estimates.

~280ms

Feature Freshness
Measured end-to-end
(<2s CONTINUOUS guarantee)

~20–40ms

End-to-End Scoring
avg · initial Zilch POC
(end-to-end incl. PrivateLink)

1

Platform
replacing 7+ independently managed AWS services

Zero

Custom ETL pipelines built or maintained

LATENCY BREAKDOWN (100 REAL TRANSACTIONS)

Component	p50	p95	p99
OFS Feature Group (5 views, 1 call)	~11ms	~11ms	~13ms
XGBoost inference	~1ms	~1ms	~1ms
Total (Snowflake internal components)	~11ms	~12ms	~14ms
Preliminary Zilch POC (end-to-end)	~20–40ms avg — well inside Visa/MC 100–200ms authorization window		

SUCCESS CRITERIA

- ✓ Feature freshness <2s — ~280ms measured
- ✓ Scoring latency — preliminary Zilch POC: ~20–40ms avg, well within Visa/MC window
- ✓ Same AWS backbone — SPCS on EC2, same region
- ✓ No ETL / No pre-processing pipeline — Zero Glue jobs
- ✓ Training/serving parity — drift structurally impossible
- ✓ Real-time path is the MVP path — no custom build required

① Snowflake-internal component timings measured on 100 real transactions (12M row table). Preliminary end-to-end testing on Zilch infrastructure shows ~20–40ms avg including PrivateLink. Both figures well inside Visa/MC 100–200ms authorization window.

Three compounding improvements from one platform decision.

LATENCY

~20–40ms avg end-to-end
preliminary Zilch POC result
including PrivateLink

60–180ms headroom
vs Visa/Mastercard 100–
200ms authorization window

**Card testing visible from
txn 2**
burst attack blocked before it
completes

~280ms feature freshness
vs minutes on batch pipelines

COST IMPACT

Meaningful reduction in fraud losses

Real-time burst detection
closes the card-testing gap
that batch pipelines miss

Scales with transaction growth

Impact compounds as volume
grows — no re-architecture
required

Engineering cost reduction

Fewer services to own, fewer
on-call incidents, faster
iteration cycles

Faster model iteration

Retrain, evaluate, and deploy
in hours not days — more
cycles per year

OPERATIONAL SIMPLICITY

1 credential surface
vs per-service IAM roles,
access keys, endpoint auth

1 network boundary
vs VPC peering, security
groups per service

1 monitoring surface
vs CloudWatch + X-Ray +
custom dashboards

No data egress
all training and serving stays
inside Snowflake

On cost impact: Specific figures to be modelled against Zilch's actual fraud rates, transaction volumes, and engineering costs as part of the formal TCO assessment in Step 2.

Zilch's real-time feature pipeline is still being built. Every week costs detections.

FEATURE FRESHNESS	WHAT THE MODEL SEES	OUTCOME
Minutes (batch pipeline)	Yesterday's velocity	Attack completes undetected. All 5 transactions approved.
30–40s (micro-batch)	Near-current activity	Catches slower attacks. Misses bursts in <30s window.
<2s — Snowflake CONTINUOUS	Real-time velocity	Burst visible from transaction 2. Attack blocked before it completes.

"Every week the pipeline remains under construction is a week the fraud model runs blind to intra-session patterns."

WHAT SNOWFLAKE DELIVERS OUT OF THE BOX

- ✓ Sub-2s freshness — no custom build, no batch cycle to engineer away
- ✓ CONTINUOUS aggregation updates on every ingest event, automatically
- ✓ Full stack deploys in a single notebook run

WHAT THIS POC DEMONSTRATES

- ✓ Built and benchmarked on live Snowflake infrastructure
- ✓ ~280ms freshness measured (not estimated) on real data
- ✓ Ready to run against Zilch's own transaction history today

Online Feature Store is in Public Preview.

Snowflake provides enterprise support terms and committed SLAs for POC engagements. Direct access to product engineering included.

Two steps to a production-grounded decision.

STEP 1

POC on Zilch's Live Transaction Data

Run the validated pipeline against a sample of real Zilch transaction history. Measure freshness improvement vs current baseline. Validate latency on your actual entity cardinalities (customer / merchant / DPAN / IP). The infrastructure template and deployment runbook are ready — this is a configuration exercise, not a build.

STEP 2

Formalised TCO and Business Value Assessment

Using Zilch's actual fraud rates, transaction volumes, and engineering costs, we convert the estimates in this deck to a business case grounded in your numbers. Snowflake account team provides Online Feature Store pricing for the Preview engagement. Output: a decision-ready cost/benefit model.

Why now: Zilch's real-time feature freshness capability is still being built. Snowflake can deliver sub-2-second freshness without waiting for a custom pipeline to ship — and without the ongoing burden of maintaining it. The technical validation is complete. The only remaining question is whether these numbers hold on Zilch's data.